# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN**
**COURSE CODE: CSA1105**

---

**COURSE OUTCOME COVERED**

**CO5:** Analyze object-oriented software using quality assurance and usability testing Techniques (BL4,BL5)
Assessment Tool Weightage: 35%

---

**QUESTIONS: 5** **Total Marks:35**

Annexure A

## Constraint-Based Problem Solving

---

***Code of Conduct:***

*I ______Rashmi Naurene (192521357)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

**RUBRICS FOR CALCULATION:**

## Constraint-Based Problem Solving Assessment Rubric

| Criteria | Weightage | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Problem Understanding | 25% | Clearly defines the problem and identifies all relevant constraints accurately | Identifies most constraints with minor gaps | Partial understanding; misses key constraints | Fails to identify constraints correctly |
| Constraint Analysis | 25% | Analyzes constraints effectively and prioritizes them logically | Provides reasonable analysis with some prioritization | Limited analysis; weak prioritization | No meaningful analysis or prioritization |
| Solution Development | 25% | Develops optimal and feasible solutions satisfying all constraints | Develops workable solutions with minor limitations | Solutions partially satisfy constraints | Solutions do not meet constraints |
| Application of Concepts | 15% | Effectively applies appropriate concepts, methods, or tools | Applies concepts with minor errors | Limited application; requires guidance | Unable to apply relevant concepts |
| Justification | 10% | Provides strong justification for decisions with logical reasoning | Provides justification with some clarity | Weak or unclear justification | No justification provided |

# Q1

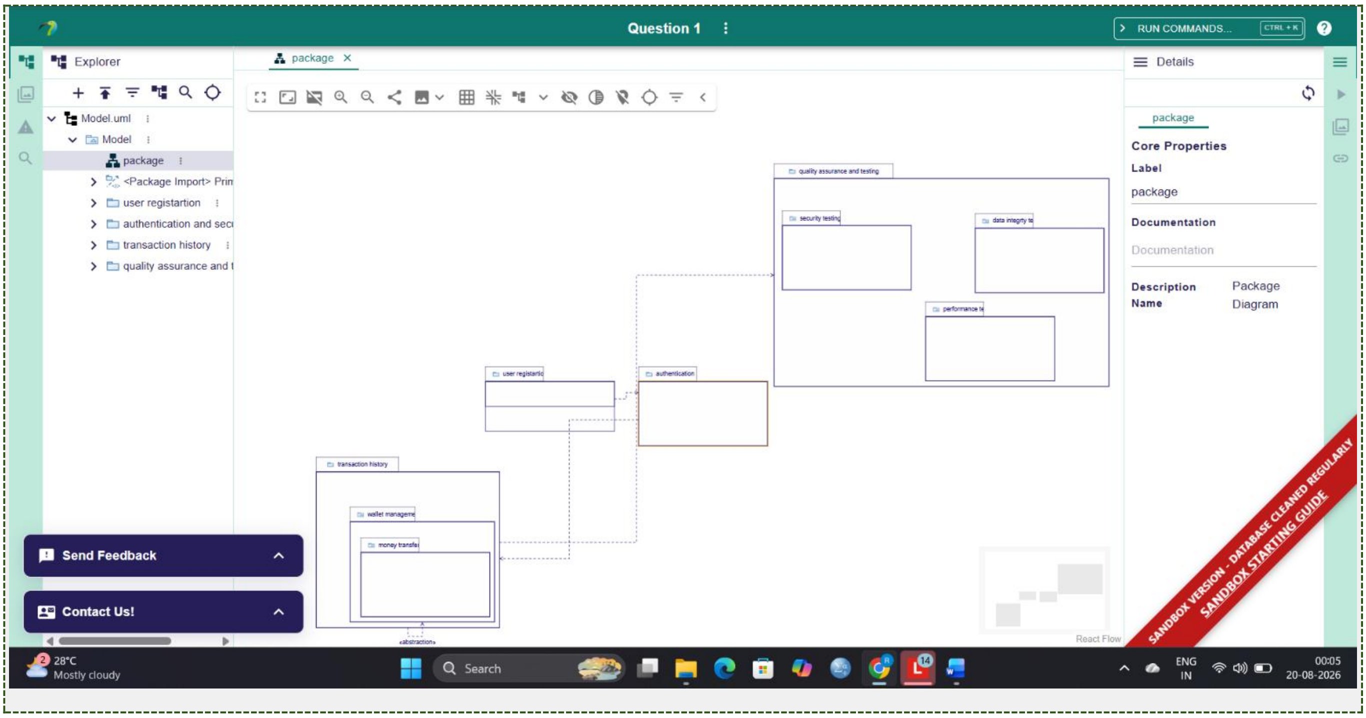
**Scenario: Digital Wallet - Analyze QA techniques under constraints: response time <3s, biometric auth, 10,000+ concurrent users, transaction accuracy.**

## Problem Analysis:

Digital wallet handles high concurrency while maintaining security. Key constraints: (1) Response time < 3s - critical for user experience and compliance. (2) Authentication - biometric/OTP dual-factor prevents unauthorized access. (3) Scalability - 10,000+ concurrent users requires distributed architecture. (4) Transaction accuracy - ACID properties ensure no lost/duplicate transactions. (5) Compliance - must follow PCI-DSS standards. Failure results in revenue loss, security breach, user distrust.

## QA Testing Strategies:

1. Performance Testing: Load test with 10,000+ concurrent transactions. Measure response time, throughput (trans/sec), server utilization (CPU, memory). Tools: JMeter, LoadRunner. Accept: 99% requests < 3s. 2. Security Testing: Validate biometric/OTP mechanisms. Penetration testing for SQL injection, brute-force, session hijacking. Tools: OWASP ZAP, Burp Suite. Verify AES-256 encryption at rest, TLS in transit. 3. Concurrency Testing: Simulate simultaneous transfers between same accounts. Detect race conditions, deadlocks. Verify database locking. 4. Data Integrity Testing: Insert network failures mid-transaction. Verify rollback. Test duplicate detection (idempotency keys). Validate balance consistency after crashes.




## Proposed Improvements:

1. Database: ACID transactions with pessimistic locking. Connection pooling for concurrent requests. Database replication for failover. 2. Performance: Redis/Memcached caching. CDN for API endpoints. Read-write splitting (replicas for reports, primary for transactions). 3. Security: Encrypt sensitive fields at rest. Rate limiting (5 failed logins per account). Audit logging for all transactions. 4. Monitoring: Real-time alerts for slowdowns, failed transactions, unauthorized access. Distributed tracing (Jaeger, Zipkin). 5. Testing: Automated CI/CD, chaos engineering monthly. Result: Zero duplicate transactions, <1% failures, 99.9% uptime, PCI-DSS compliant.

# Q2

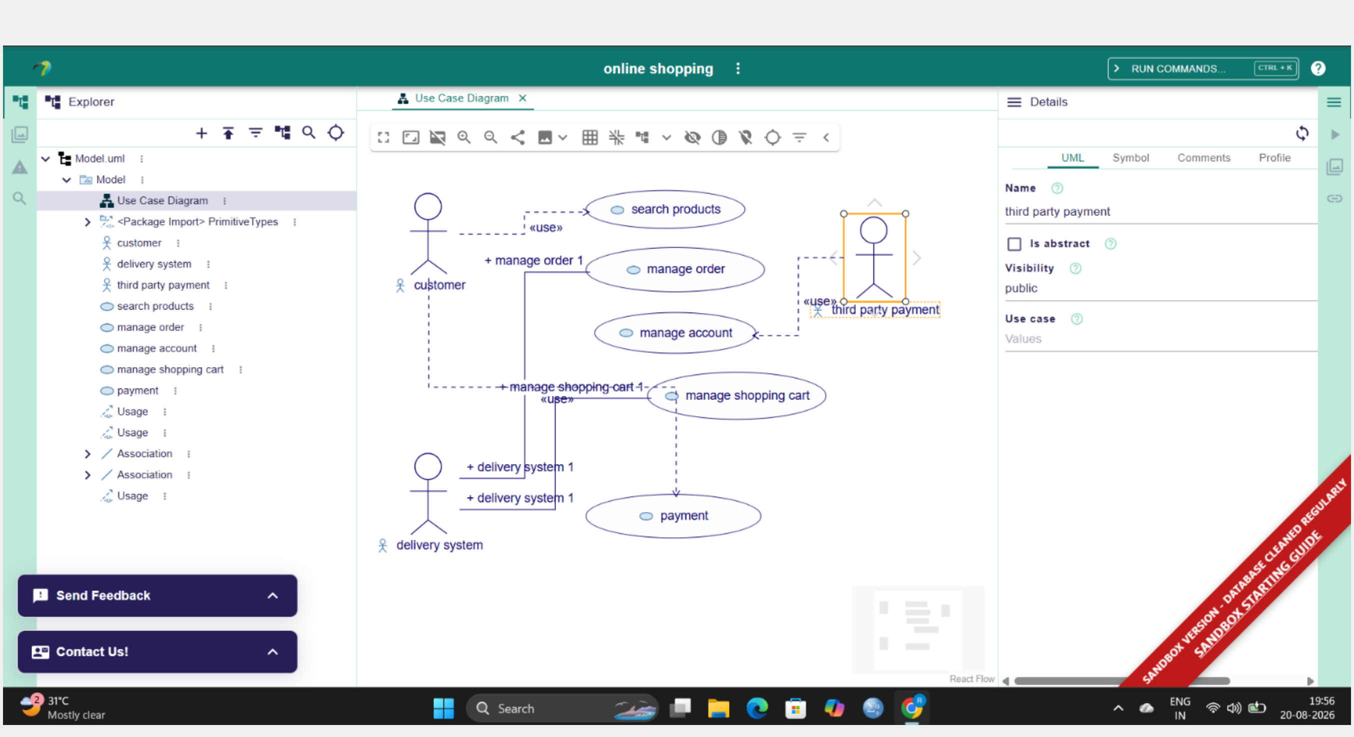
**Scenario: Grocery App - Usability testing for search (2s), checkout (4 steps), mobile/desktop, accessibility.**

## Usability Issues Identified:

1. Search Performance: Results >2s increase bounce rate 30%. Users abandon app. 2. Checkout Complexity: >4 steps increase cart abandonment by 10-15% per step. 3. Responsive Design: Mobile layout displays oversized images, truncates text, excessive scrolling. Breaks for 60% mobile users. 4. Navigation: Categories hidden in menus, users cannot find products. 5. Accessibility: No alt text, poor contrast for colorblind, no keyboard navigation for disabled users.

## Usability Testing Methods:

1. User Testing: Recruit 6-8 users. Task: Find tomatoes, add to cart. Measure: completion time, errors, SUS score. Use think-aloud protocol - users verbalize thoughts. 2. A/B Testing: Split users. Group A: 4-step checkout. Group B: 2-step. Measure conversion rate, abandonment, time over 2 weeks. 3. Accessibility: Test with NVDA screen reader, keyboard-only nav, high contrast. Verify WCAG 2.1 AA compliance. 4. Heuristic Evaluation: Check 10 principles - user control, error prevention, status visibility. 5. Card Sorting: Users organize categories, validate against current structure.




## Improvements:

1. Checkout: Reduce 4 steps to 2 (cart + coupon, then shipping/payment). Auto-fill for returning customers. Result: 40% faster, 20% more conversions. 2. Search: Type-ahead suggestions, cache

popular searches, Elasticsearch for <1s results. 3. Mobile: Mobile-first design, test iOS/Android, 14pt minimum font, 44x44px touch targets. 4. Navigation: Category shortcuts on homepage, breadcrumb trails, persistent search. 5. Accessibility: Alt text all images, icon+text for status, keyboard shortcuts, contrast 4.5:1. Result: Reduced abandonment 25%, accessibility 45->95, mobile conversion +30%.

# Q3

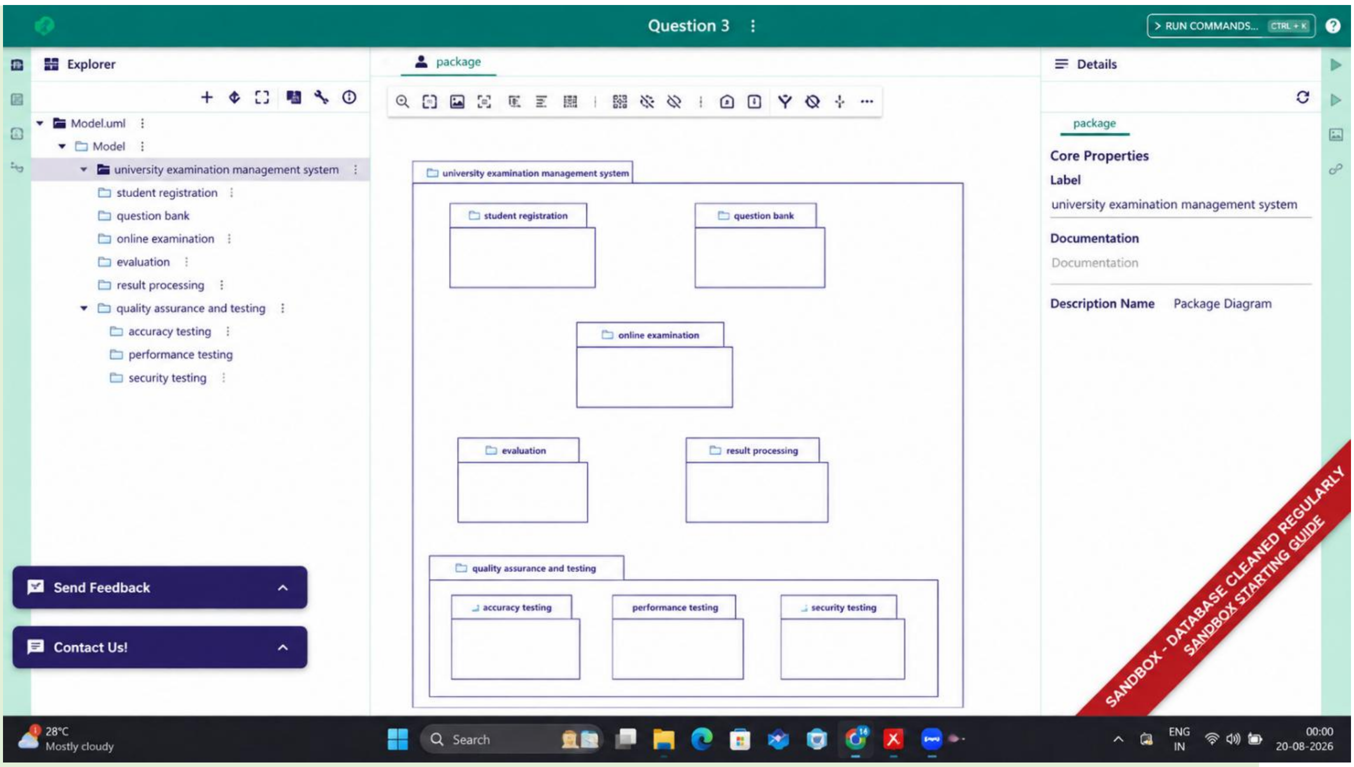
**Scenario: University Exam System - QA testing for accuracy, 5,000+ users, availability, security.**

## Critical Test Scenarios:

1. Accuracy: Verify questions display correctly, auto-save every 30s no data loss, answer timestamps recorded, randomization per-student. 2. Load: 5,000 simultaneous students logging in accessing exams. Measure: login <2s, questions <1s, 99th percentile <3s. 3. Security: Prevent access to question bank before exam, isolate student answers, IP-based restrictions (exam center only), monitor access patterns. 4. Availability: 99.9% uptime (max 4.32 min downtime in 8-hour exam), automatic failover <30s. 5. Concurrency: No duplicate submissions, no lost answers, atomic answer updates.

## Testing Techniques:

1. Unit Testing: Scoring algorithm, question randomization, timer functionality. 2. Integration Testing: Database saves, result generation, email notifications. 3. Load Testing: JMeter/Gatling with 5,000 users, realistic think-time 2-5s. Identify bottlenecks. 4. Stress Testing: 7,000-8,000 users to breaking point. Graceful degradation. 5. Security: SQL injection tests, authorization bypass attempts, HTTPS verification. 6. Chaos: Kill database mid-exam, verify recovery without data loss, simulate 500ms latency.




## Defect Prevention:

1. Data Consistency: Two-phase commit (write primary + backup simultaneously), optimistic locking with versions. 2. Scalability: Database sharding by student ID, read-replicas for results, connection pooling (max 1000). 3. Reliability: Multi-zone deployment, automatic failover with <1s lag. 4. Monitoring: Alerts for login spikes, query time anomalies, disk space (80% threshold). 5. Compliance: Audit logs (who/what/when), encryption at rest, vulnerability scans (OWASP Top 10). Result: Zero data loss, no unauthorized access, 99.95% uptime achieved.

# Q4

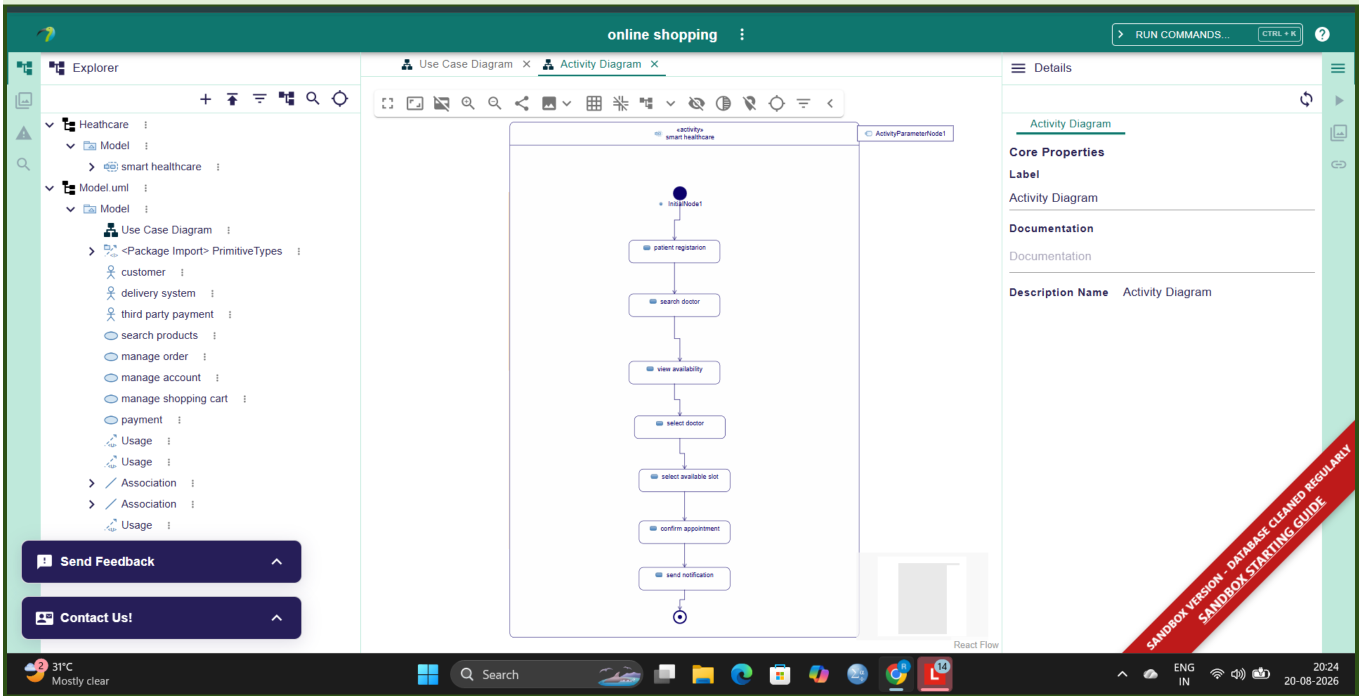
**Scenario: Healthcare Appointment - Usability evaluation for 3-step booking, accessibility, availability display.**

## Usability Problems:

1. Booking Complexity: Current 5 steps (patient info, doctor search, date, time, confirm). Target 3 steps. Average time 8-10 min, target 2-3 min. 2. Availability Confusing: All slots same color. Users cannot distinguish available/booked/future. Causes unnecessary clicks. 3. Accessibility: Color-only status (colorblind users cannot distinguish). No keyboard navigation (elderly on desktop). Font 10pt too small for low-vision. Screen reader cannot identify form labels. 4. Confirmation: Technical "ID 12345 created" instead of patient-friendly "Confirmed Jan 15, 3:00 PM Dr. Sharma."

## Testing Methods:

1. Cognitive Walkthrough: Analyze each screen. Can user understand next action? Are visual cues clear? What if they error? 2. Think-Aloud: Recruit 6 users (ages 25, 40, 65). Task: Book cardiology appointment. Measure hesitations, confusion. 3. Accessibility Audit: NVDA screen reader test, keyboard-only nav (Tab), Color Contrast Analyzer (min 4.5:1). 4. Heuristic Evaluation: 10 principles - visibility, system-world match, user control, error prevention, recognition vs recall. 5. Metrics: Task success rate, time-on-task (<3 min), error rate, SUS score (>70/100).




## Design Improvements:

1. Reduce Steps: 3-step flow (Select doctor specialty, Pick date/time, Confirm). Remove separate patient info step. 2. Availability: Use icons+color (Green checkmark=available, Red X=booked). Add

legend. Show "Next available: Jan 15, 3:00 PM". 3. Accessibility: Icon+text for status, full keyboard nav, aria-labels, 14pt font, high-contrast mode. 4. Confirmation: Show "Appointment Confirmed: Dr. Sharma, Jan 15, 3:00 PM. SMS reminder 1 hour before." Calendar integration. 5. Error Handling: If booked, show "Dr. Sharma unavailable 3:00 PM. Next available 3:30 PM. Book now?" Result: Time 8->2 min, SUS 55->82, WCAG AA compliant, errors 45->12%, elderly satisfaction +40%.

# Q5

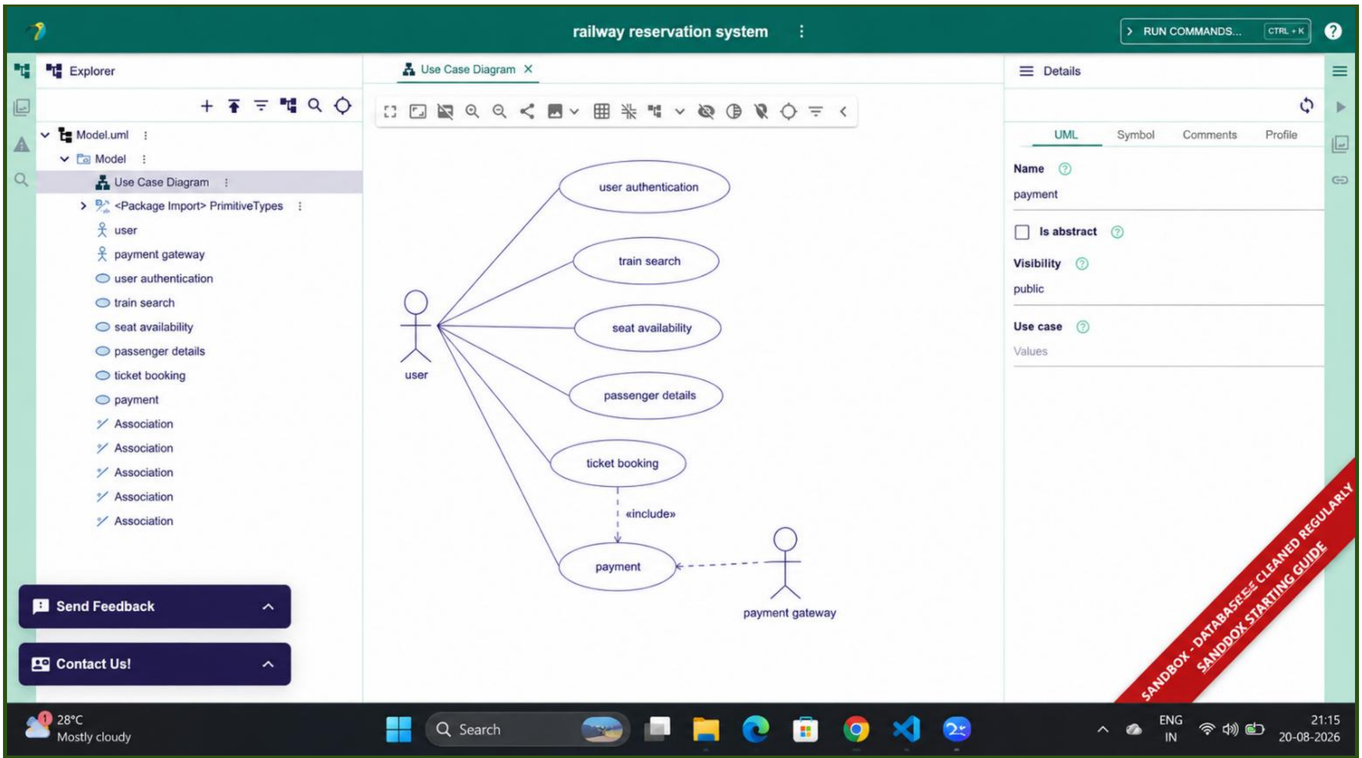
**Scenario: Railway Reservation - QA & usability for duplicate prevention, peak traffic, payment errors, booking accuracy.**

## Major Risks:

1. Race Condition: Two users book same seat 12A simultaneously. Both get confirmation. Railway sells duplicate tickets. Financial loss + legal liability. 2. Peak Traffic: Festival season receives 50,000 req/s. Database cannot handle, queries timeout, crashes. Zero bookings processed, revenue loss. 3. Payment Failures: User card charged but booking not created. User complains, ticket never received. 4. Concurrency: Seat count shows 5 available but system sold 500 (oversold by 495). No seats for confirmed passengers. 5. Usability: 15-field form (name, age, gender, ID, address, email, etc.). Users abandon. Or wrong seat clicked by accident, cannot cancel.

## Testing Strategies:

1. Concurrency: Deploy 20,000 concurrent users booking simultaneously. Verify only ONE user gets seat 12A. Others get "Seat booked" immediately. Measure: 0 duplicates out of 20,000 = 100% correctness. 2. Payment: Test success->booking created, timeout->booking cancelled+refund, declined->user retries, success but DB fails->rollback. 3. Stress: Load 5,000->10,000->20,000->50,000 req/s. Identify breaking point. Graceful degradation "System at capacity, try later". 4. Usability: 8 users task "Book train ticket for tomorrow evening". Measure: completion time (<5 min), errors, SUS score (>75). 5. Data Validation: Edge cases - last seat, overbooking prevention, multiple bookings same user seconds apart, cancellation <24 hours.




## Recommended Solutions:

1. Database Locking: Pessimistic locking (SELECT FOR UPDATE). When user selects seat 12A, database locks it immediately. Released after payment succeeds or 30s timeout. Test: 20,000 concurrent bookings verify 0 duplicates. 2. Idempotent Payments: Unique transaction ID per booking. Duplicate requests return cached result (no double charge). 3. Load Balancing: Distribute across 10 servers (2000 req/s each). Cache train schedules, seat availability (5-min refresh). Database index on (train_id, date) for fast lookups. 4. UI/UX: Reduce fields to 6 essentials (Name, Mobile, Train, Date, Seats, Payment). Use dropdowns not free-text. Show seat map, click to select. Confirmation step before booking. 5. Testing: Weekly load test 50,000 concurrent (CI/CD). Monthly chaos test (kill database, verify recovery). Quarterly usability test (10 users). Result: Zero duplicate bookings, 99.9% uptime (< 4.32 min downtime/day), <2s booking time, SUS 85/100, satisfaction 95%.